Instance based Learning

Instance based Learning

- In contrast to learning methods that construct a general, explicit description of the target function when training examples are provided.
- Instance-based learning methods simply store the training examples.
- Generalizing beyond these examples is postponed until a new instance must be classified
- Each time a new query instance is encountered, its relationship to the previously stored examples is examined in order to assign a target function value for the new instance

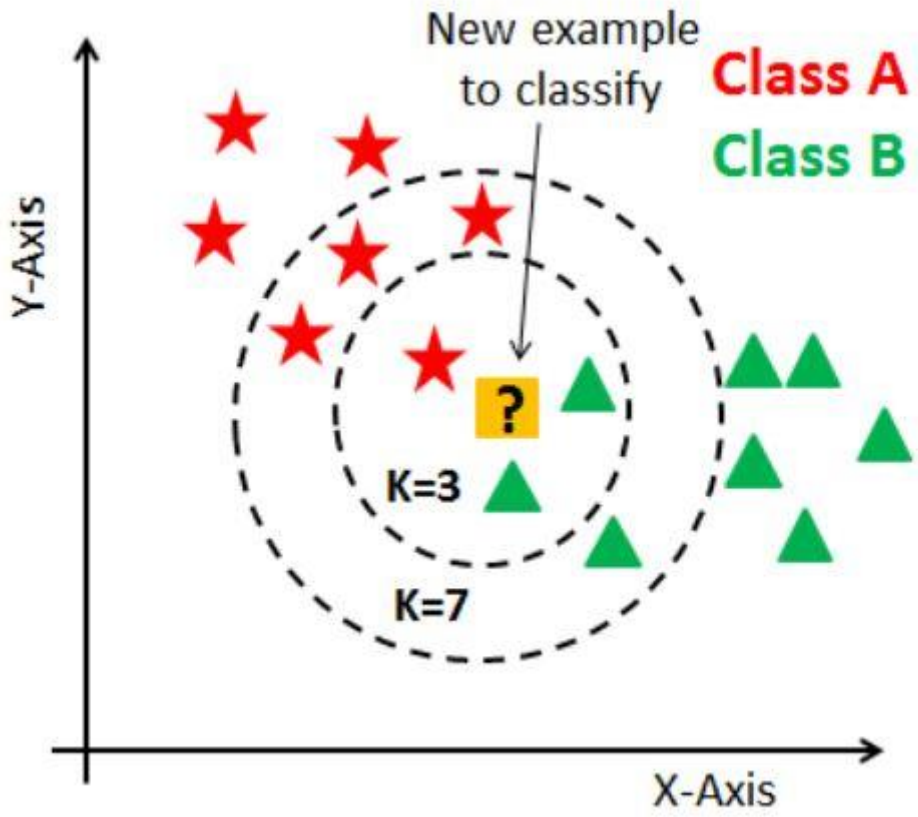
Example: KNN

This kind of learning is also called Lazy Learning.

K Nearest Neighbor

- Introduction
- Metrics
- Selection of K
- Design Issues
- Advantages and Drawbacks

Introduction



K-Nearest Neighbor

- Simplest Machine Learning algorithms based on Supervised Learning technique.
- K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data.
- Does not make any assumptions on the data.
- Instance based learning or Lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset.
- K-NN algorithm computes the similarity between the new data and available data and put the new data into the category that is most similar to the available categories.
- K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

Procedure

The K-NN working can be explained on the basis of the below algorithm:

1. Select the number of the nearest neighbors
2. Calculate the similarity of the query data point with neighbors.
3. Compute the **K number of nearest neighbors** w.r.t to query point.
4. Assign the new data points to that category for which the number of the neighbors are maximum.

How to Choose Parameters

Parameters in KNN:

1. # of Nearest Neighbors
2. Similarity Measure

Selection of K

- There is no standard procedure to select the K.
- In practice, we prefer K= 3 or 5
- What if K=1 ?
- Select K using the Cross validation:

K	Error
1	0.35
2	0.30
3	0.24
4	0.29
5	0.4

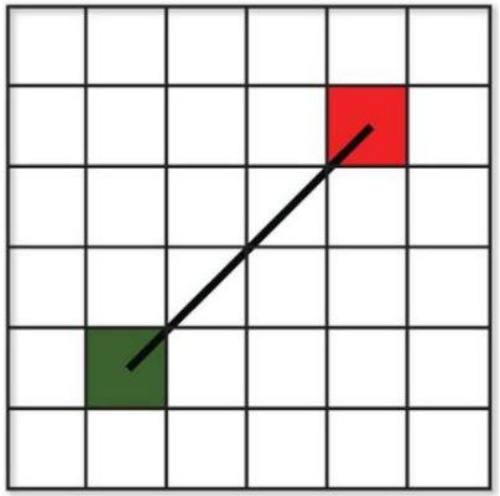
Similarity Metrics

Widely used similarity metrics are:

- Manhattan Distance
- Euclidian Distance
- Cosine Distance
- Jaccard Distance
- Hamming Distance

Euclidean distance

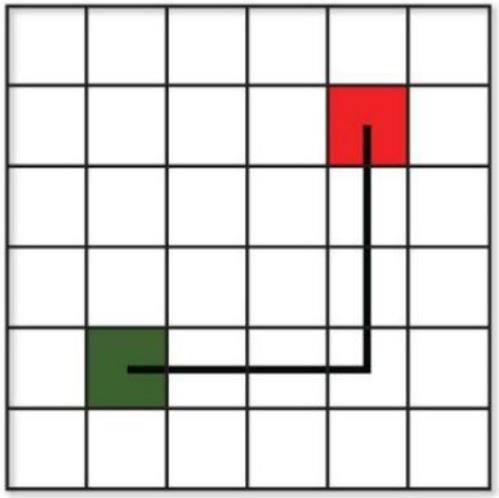
Widely used distance metric. It is a measure of the true straight line distance between two points in Euclidean space.



$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Manhattan Distance

Also Called taxi cab or city block distance



$$d = \sum_{i=1}^{n} |x_i - y_i|$$

In case of high dimensional features, we prefer Manhattan Distance

Cosine Distance

- This distance metric is used mainly to calculate similarity between two vectors.
- Measures by the cosine of the angle between two vectors and determines whether two vectors are pointing in the same direction.
- It is often used to measure document similarity in text analysis.

$$\cos\theta = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \cdot \|\vec{b}\|}$$

Jaccard Distance

The Jaccard approach looks at the two data sets and finds the incident where both values are equal

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

Hamming Distance

- Hamming distance is a metric for comparing two binary data strings
- Hamming distance is the number of bit positions in which the two bits are different

KNN Work Flow

Training Data (X)	Features				Label (y)
x1	a11	a12	a13	a14	red
x2	a21	a22	a23	a24	red
x3	a31	a32	a33	a34	red
x4	a41	a42	a43	a44	red
x5	a51	a52	a53	a54	red
x6	a61	a62	a63	a64	Green
x7	a71	a72	a73	a74	Green
x8	a81	a82	a83	a84	Green
x9	a91	a92	a93	a94	Green
x10	a01	a02	a03	a04	Green

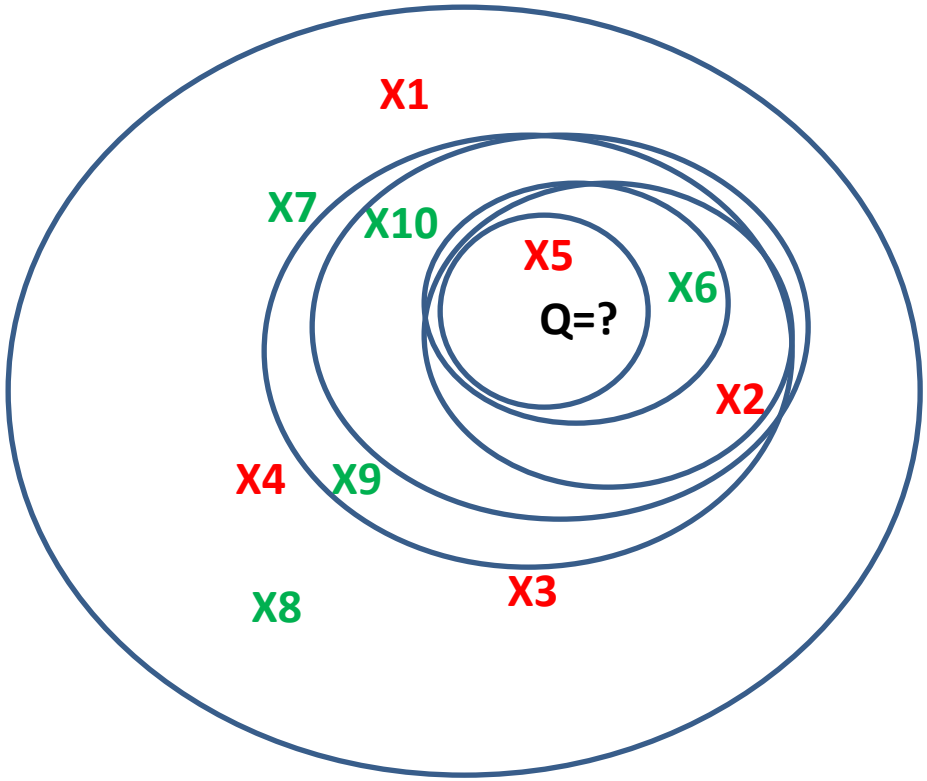
Query Data					
Q	ax1	ax2	ax3	ax4	?

Sort the training data based on the high similarity with Q

<X^i, Q> : Gives the similarity between X^i and Q.

x5	a51	a52	a53	a54	red
x6	a61	a62	a63	a64	Green
x2	a21	a22	a23	a24	red
x10	a01	a02	a03	a04	Green
x9	a91	a92	a93	a94	Green
x7	a01	a02	a03	a04	Green
x1	a11	a12	a13	a14	red
x8	a81	a82	a83	a84	Green
x3	a31	a32	a33	a34	red
x4	a41	a42	a43	a44	red

K Nearest Neighbors Representation



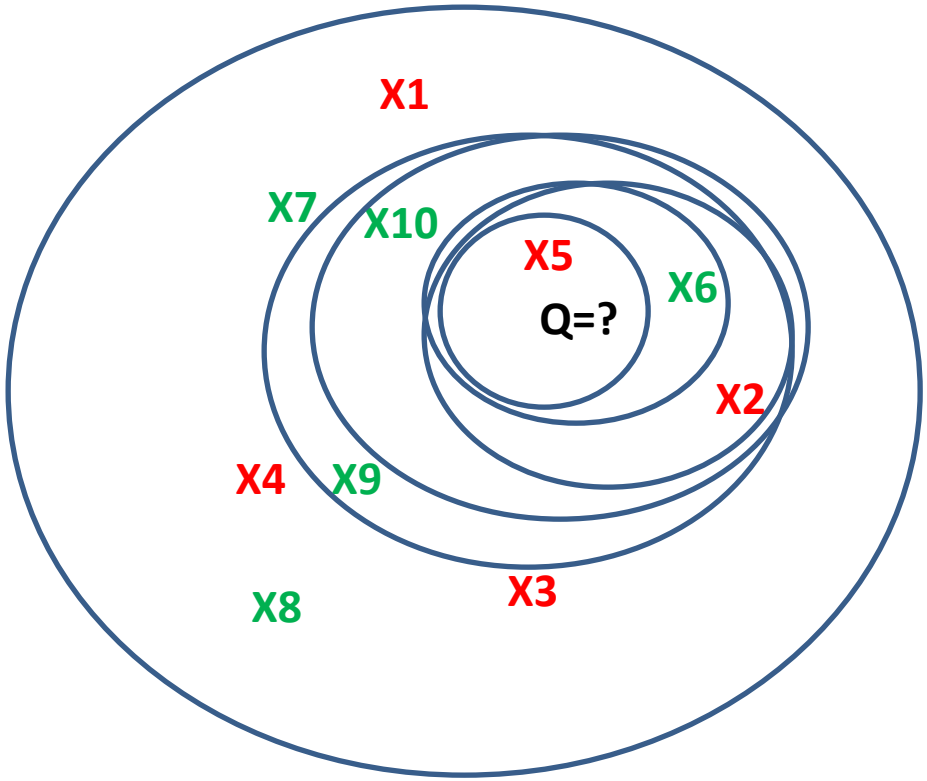
- Pick top 5 Training data, count number of votes obtained for each category.
- Assign label to the Query data based on the label which got maximum votes

x5	a51	a52	a53	a54	red
x6	a61	a62	a63	a64	Green
x2	a21	a22	a23	a24	red
x10	a01	a02	a03	a04	Green
x9	a91	a92	a93	a94	Green
x7	a01	a02	a03	a04	Green
x1	a11	a12	a13	a14	red
x8	a81	a82	a83	a84	Green
x3	a31	a32	a33	a34	red
x4	a41	a42	a43	a44	red

Query Data

Q	ax1	ax2	ax3	ax4	**Green**

K Nearest Neighbors



Advantages and Disadvantages

Advantages of KNN Algorithm:

- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.

Disadvantages of KNN Algorithm:

- High Computational Cost
- Does not work if the feature dimensions are high

Some case studies

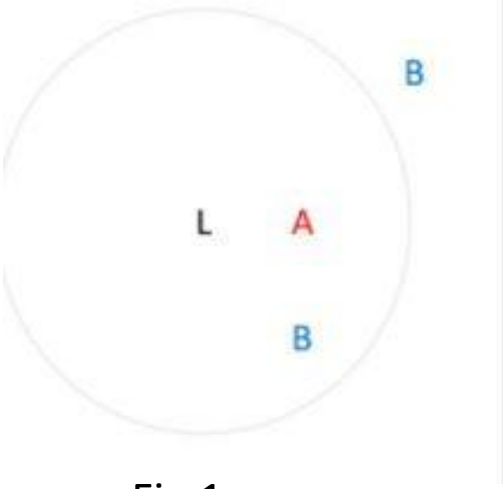


Fig.1

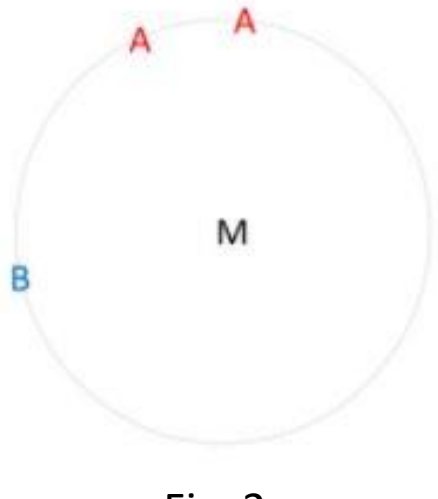


Fig. 2

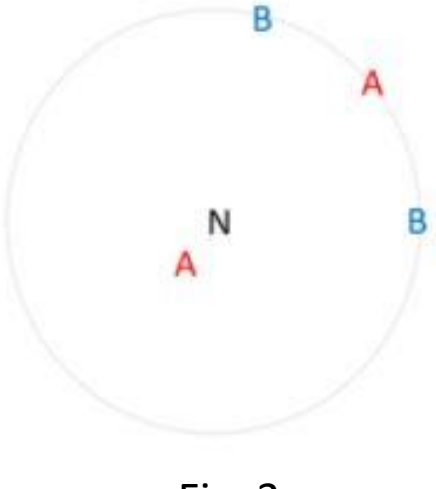
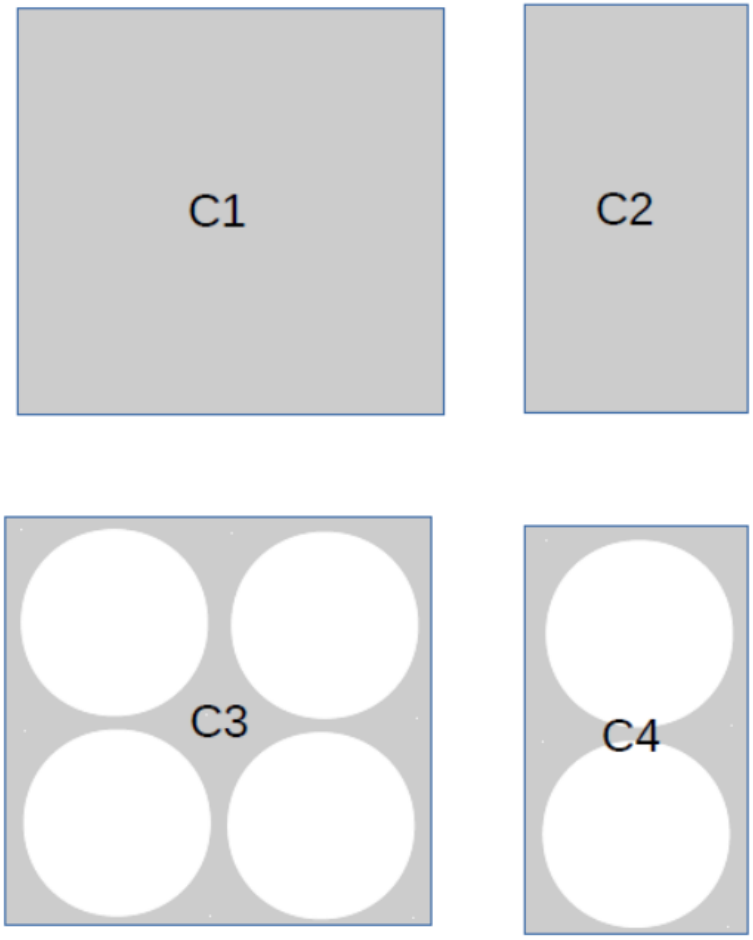


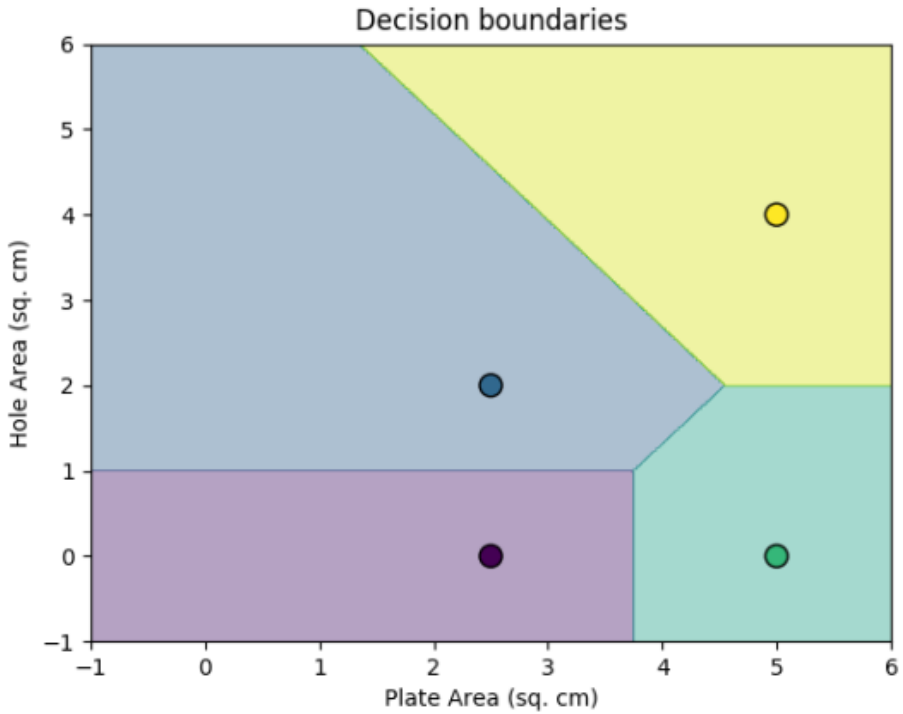
Fig. 3

- *Choose a different k*
- *Randomly choose between the tied values.*
- *Allow observations in until natural stop point*

Decision Boundaries



Steel-plate classification



Decision boundaries are perpendicular bisectors of the lines joining nearest competitors!

k-Nearest Neighbor (k-NN) Regression

- Regression is a supervised learning problem
- In regression, the target variable admits continuous values

S.No	1	2	3	4	5	6	7	8	9	10	t
x	5.2	4.8	5.8	5.7	5.4	5.1	4.9	5.3	5.9	4.5	5.6
y	58	53	63	65	60	50	52	55	61	49	?

- Regression is a supervised learning problem
- In regression, the target variable admits continuous values

S.No	1	2	3	4	5	6	7	8	9	10	t
x	5.2	4.8	5.8	5.7	5.4	5.1	4.9	5.3	5.9	4.5	5.6
y	58	53	63	65	60	50	52	55	61	49	?

- Average of the k-nearest neighbors in the training data.
- $(k = 1)$ NN - $\{(5.7, 65)\}$: $y_{11} = 65$ (overfit - high variance)
- $(k = 3)$ NN - $\{(5.7, 65), (5.8, 63), (5.4, 60)\}$: $y_{11} = 62.66$
- $(k = 10)$ Assigns average weight irrespective of test height
- Equal importance to all neighbors irrespective of proximity to test point

Inverse Distance Weighted k-NN

- Weighted k-NN regression: the unknown quantity y_t is estimated as

$$y_t = \frac{\sum_{n=0}^{N} k(x_n, x_t) y_n}{\sum_{n=0}^{N} k(x_n, x_t)}$$

Inverse Distance Weighted k-NN

- Weighted k-NN regression: the unknown quantity y_t is estimated as

$$y_t = \frac{\sum_{n=0}^{N} k(x_n, x_t) y_n}{\sum_{n=0}^{N} k(x_n, x_t)}$$

- $k(x_n, x_t)$ denotes similarity between x_n and x_t
- Similarity (or inverse distance) as weight to give preference to NNs
- $(k = 3)$ NN - $\{(10, 65), (5, 63), (5, 60)\}$: $y_{11} = 63.25$
- Number of neighbors k is not very critical.
- Similarity measure $k(x_n, x_t)$ is an important design choice

KNN Summary

- **Pros**
  - Simple to implement
  - Flexible to feature/distance function choices (no differentiability)
  - Naturally handles multicalss data
  - Does well even in most complicated cases with enough data
- **Cons**
  - Needs to access entire training data for every inference
  - Computational complexity grows linearly with training data $O(ND)$
  - Cannot naturally ignore noninformative dimensions
  - Distance function has to be chosen carefully
  - The choice of k is empirical (no theoretical guarantees)
  - Does not consider all the data at once, and hence it cannot model the underlying process responsible for the observed data

Additional Reading

Textbook: Machine Learning by Tom Mitchel, Chapter:8